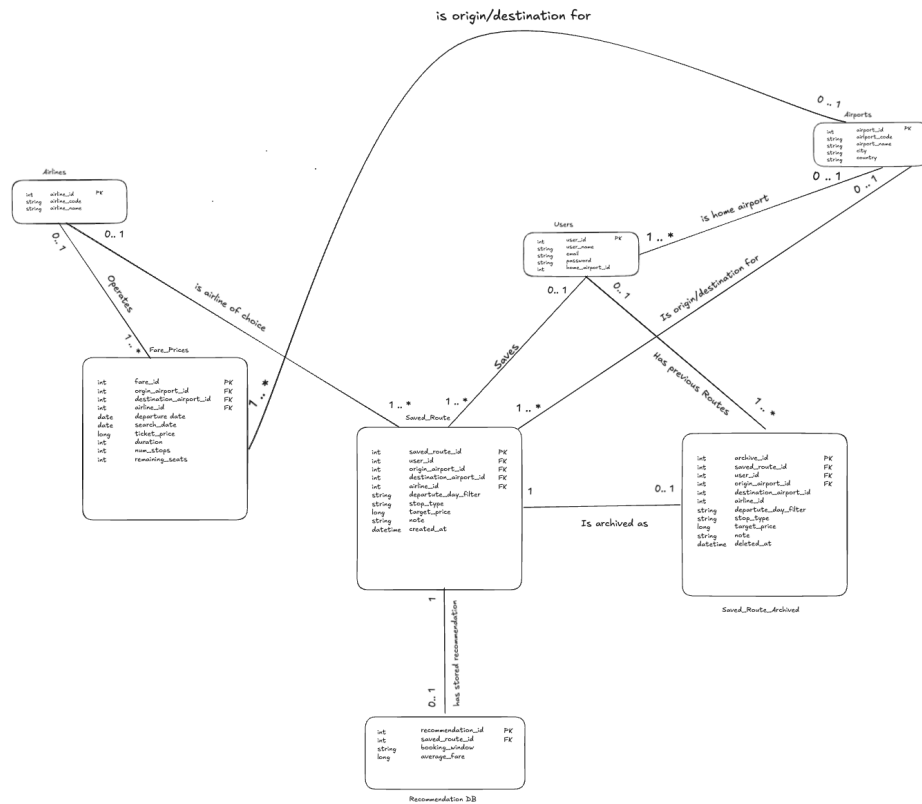


## UML Diagram



[Link to Diagram](#)

### Assumption for Entities and Relationships

**Users:** The users entity represents each person who creates an account on our platform. A user will have information such as username, email, password, and home airport. We chose to model users as their own entity because each user can have their own saved routes, recommendations and route history. As a result these entities can't have attributes of another entity. Our application will be able to create accounts where they can later update profile information.

**Airports:** The Airports entity stores information about airports such as code, airport name, city and country. Airports are modeled as a separate entity instead of storing the airport name within other routes because the same airport can be referenced by many users and

have relationships to other saved routes and fare records. This avoids having to store the same airport information multiple times. This will allow users to search for airports directly from the table.

**Airlines:** The airlines entity stores information about each airline storing the name, id and code. It is a separate entity because the airline can occur in many other historical fare records and can be selected as a filter for saved routes. Airlines also have many relationships within routes and airports. Airline information is part of historical datasets and can also be used as a filter.

**Saved Route:** The saved route entity represents a route and set of preferences that a user chooses to save. A user will be able to store them to show previous search histories and routes to compare each other. It stores values such as origin, destination, airline preference and departure day to see best prices. This is an entity rather than an attribute of Users because one user can save multiple routes and each saved route can have its own set of parameters making it better to store as an entity rather than an attribute.

**Saved Route Archived:** The Saved Route Archived entity stores routes that a user previously saved but later removed from their active routes. This will be updated with a Trigger when deleting from the saved routes table. We use a separate entity so that active routes and historical routes will remain separate allowing them to scale separately.

**Recommendation DB:** This entity stores calculated recommendations associated with a saved route, such as the best booking window and average fare for the parameters selected by the User. We keep this as a separate entity because each time it will be calculated from historical fare and derived data. Keeping them separate also allows the recommendation to be recalculated without changing the user's saved route.

## **Relationship Assumptions and Cardinalities**

**Users - Airports:** We chose to have 1 to many relation here because each user has one home airport while the same airport can be the home airport for zero or many users

**Users - Saved\_Route :** This is a one to many relationship because one user can have many saved routes while each saved route only belongs to one user. There are no saved routes when a user is first created but they can save multiple routes later.

**Airlines - Saved\_Routes:** One to many relationships, a saved route may have 0 or one airline selected as filters. The same airline can be selected by many saved routes. The airline can have 0 because the user can choose not to put a filter on the saved routes.

**Airports - Saved\_route:** One to many relationships, each saved route has one origin airport, while one airport can be the origin for many saved routes. This is the same relationship for both the origin and destination airports.

**Airlines - Fare\_Prices:** One to many, each fare is associated with one airline flight path while one airline can be used across many different routes, dates and prices.

Airports - Fare\_Prices: Same with saved routes there is a one to many relationship between the destination and origin airports because each trip will originate from one airport and land in one airport but each airport can be associated with many different trips.

Saved\_Route - Recommendation\_DB: This is a one to one relationship because a saved route will have the best recommendation based on the parameters that are set for the saved route. A route may temporarily not have a recommendation while the calculation is being performed. Each recommendation will belong to exactly one saved route.

Saved\_Route - Saved\_Route\_Archived: This is another one to one relation because each saved route will have exactly one archived in the database used for historical records for each user. While the route is active, no archived version exists and once the user removes the route the archived record will be created.

Users - Saved\_Route\_Archived: This is a one to many relationship because a user will have many archived routes but each archived route will be associated with exactly one user very similar to have saved\_route to user relationship.

## **Database Normalization**

Our database schema is normalized to 3NF. We decided to separate the database into different entities. Each table stores information about one specific type of object while also making sure unnecessary data duplication isn't caused. Our database satisfies 1NF because each attribute contains a single value. We did this to try and avoid repeating groups. For example, the Airports table stores one airport code, airport name, city, and country. We also designed our relations to satisfy 2NF, making sure each table uses a primary key to uniquely identify each record, and that all non-key attributes depend on it. For example, in the Airlines table, the airline code and airline name depend on airline\_id. Our schema also satisfies 3NF because non-key attributes depend only on the primary key, and not on attributes that aren't keys. We separated information into different entities to prevent duplication amongst different tables. For example, airline information is stored in the Airlines table, while Fare\_Prices only stores the airline\_id as a foreign key. Another example of how we implemented this is how airport information is stored in the Airports table and can be referenced by Saved\_Route and Fare\_Prices through using foreign keys. This normalization helps us to optimize update, insertion, and deletion calls. For instance if an airline's name changes we only need to update it in the Airlines table. We can also add an Airport to the Airports table without requiring a saved route or fare record. Through this design, our database satisfies 3NF because each relationship represents a specific entity, each non key attribute depends on its relation's primary key, and because information is not duplicated throughout the tables.

## **Relational Schema**

Users(user\_id:INT [PK], username:VARCHAR(50), email:VARCHAR(255), password\_hash:VARCHAR(255), home\_airport:VARCHAR(3) [FK to Airports.airport\_code])

Airports(airport\_code:VARCHAR(3) [PK], airport\_name:VARCHAR(100), city:VARCHAR(100), country:VARCHAR(100))

Airlines(airline\_id:INT [PK], airline\_code:VARCHAR(3), airline\_name:VARCHAR(100))

Fare\_Prices(fare\_id:INT [PK], airline\_id:INT [FK to Airlines.airline\_id], origin:VARCHAR(3) [FK to Airports.airport\_code], destination:VARCHAR(3) [FK to Airports.airport\_code], booking\_date:DATE, departure\_date:DATE, fare:DECIMAL(8,2))

Saved\_Route(route\_id:INT [PK], user\_id:INT [FK to Users.user\_id], origin:VARCHAR(3) [FK to Airports.airport\_code], destination:VARCHAR(3) [FK to Airports.airport\_code], airline\_id:INT [FK to Airlines.airline\_id], departure\_date:DATE, created\_at:DATETIME)

Saved\_Route\_Archived(route\_id:INT [PK], user\_id:INT [FK to Users.user\_id], origin:VARCHAR(3) [FK to Airports.airport\_code], destination:VARCHAR(3) [FK to Airports.airport\_code], airline\_id:INT [FK to Airlines.airline\_id], departure\_date:DATE, created\_at:DATETIME, archived\_at:DATETIME)

Recommendation\_DB(route\_id:INT [PK] [FK to Saved\_Route.route\_id], best\_booking\_window:INT, average\_fare:DECIMAL(8,2), last\_calculated:DATETIME)